Nemu: A Cross-Platform Emulator for the Nintendo Entertainment System Architecture

Lucaz Lindgren

2025-08-13

Contents

1 Abstract

The primary goal of this project was to develop Nemu, a cross-platform software emulator for the Nintendo Entertainment System (NES), allowing users to experience the vast library of classic NES games on modern computer systems.

Nemu was developed as a minimalist, terminal-based emulator using Raylib and the Odin programming language with emphasis on emulating the core hardware components. These include the Ricoh RP2A03 chip containing the 6502-based CPU and custom Audio Processing Unit (APU), and the Ricoh 2C02 Picture Processing Unit (PPU). This makes the emulator capable of emulating the NTSC version of the NES console and, by extension, the games developed for it (PAL compatibility is completely omitted).

The resulting emulator successfully passes key CPU instructions tests for established diagnostic ROMs. It handles graphics rendering, input handling, and audio synthesis with minor inaccuracies for a significant portion of games released early in the NES's life cycle. Notably, it does not contain a Graphical User Interface (GUI), save state system, gamepad remapping, or many of the other features often expected of emulators. That development time has instead been allocated to emulation accuracy and game compatibility. This keeps the project small and focused at the expense of usability. The emulator is distributed as a single statically linked binary, meaning that it has no external runtime dependencies. This makes it trivial to download and run on any supported system.

This project successfully demonstrated a comprehensive understanding of low-level hardware architecture and common emulation techniques, resulting in a robust application capable of preserving the full experience of a classic game console.

2 Introduction

2.1 Background

The Nintendo Entertainment System (NES) is an 8-bit home video game console first released in the North American market in 1985. It is widely regarded as one of the most significant video game consoles ever produced, and its launch is frequently credited with revitalizing the American video game industry following the market crash of 1983. The console has become a popular target for emulation due to its influential legacy and simple, yet elegant, design.

The core system architecture of the NES is composed of three logical components: the Central Processing Unit (CPU), the Picture Processing Unit (PPU), and the Audio Processing Unit (APU). Apart from the core system, the game cartridges themselves, and the memory mappers contained within them, play a vital role. Together, they are responsible for mapping large portions of both the CPU and PPU address spaces to various ROM and RAM chips contained within the cartridge. This flexible design allowed later NES games to be far more complex with more ROM/RAM capacity and additional APU channels for more advanced audio synthesis.

Due to different analog television standards—namely NTSC and PAL—there exist multiple versions of the NES depending on the region where it was released. North America and parts of Asia received the NTSC version of the NES, whereas Europe, Africa, and parts of Asia received the PAL version of the NES. Due to diverging timing requirements between the standards, different chips were used for the PPU and APU implementations. Because Nemu only supports the NTSC version, this report will only discuss that version.

2.2 Motivation

My motivation for this project came from my interest in embedded software development. The development of a game console emulator presented a compelling challenge, as it requires a broad set of engineering skills, from understanding detailed hardware behavior to designing a cohesive system architecture. I was particularly interested in the difficulty of translating asynchronous hardware behavior into a synchronous software model. The Nintendo Entertainment System was selected as the target platform due to its historical significance, well-documented architecture, and manageable complexity.

2.3 Scope

The scope of this project was centered around the implementation of the core emulation logic required for game execution. The boundaries of the work are defined as follows:

2.3.1 In Scope

- The primary goal was to achieve a high degree of compatibility with the official Nintendo Entertainment System game library.
- Support was implemented for the iNES 1.0 and iNES 2.0 file formats.
- The emulation core successfully supported the five most common memory mappers: NROM, MMC1, UxROM, CNROM and MMC3.
- A basic debugging interface was developed and used internally to aid in the development and verification process.

2.3.2 Out of Scope

- Complete emulation accuracy was not a goal.
- Advanced user-facing features were intentionally excluded to maintain focus on the core emulation. These features include a graphical user interface (GUI), save/load states, and controller input remapping.
- Support for the less common iNES header formats, iNES 0.7 and Archaic iNES, is not supported. Neither are alternative ROM file formats such as UNIF.

3 System Architecture

A prominent challenge when emulating the NES console is ensuring the correct execution frequency of the different system components. Predominantly, the CPU, APU, and PPU. These are all synchronized by a common crystal oscillator operating at 21.477272 MHz. Frequency dividers are used to clock the components at various lower frequencies.

Component	Frequency	Division
CPU	1.79 MHz	Base/3
PPU	5.37 MHz	Base
APU	1.79 Mhz	Base/3

Table 1: Displays the different system components' operation frequencies. Base refers to the global system clock operating at 5.37 MHz (one-fourth of the crystal oscillator).

The emulator uses an internal cycle counter to maintain the correct frequency relationship between the NES components. This counter simulates the master clock of the console. With each cycle, the emulator determines which components to "tick" forward, ensuring the PPU runs at its proper 5.37 MHz and the CPU/APU at 1.79 MHz, preserving the crucial 3:1 speed ratio between them.

Nemu utilizes a multi-threaded architecture to manage its core components. The workload is divided across three main threads:

- Main Thread: Responsible for rendering the user interface (UI) and displaying the final video output from the emulator.
- Emulation Thread: Executes the primary emulation loop, running the NES's CPU, PPU, and APU cycles.
- Audio Thread: Managed by the Raylib library to handle real-time audio sample submission to the operating system.

Correct audio timing is critical in emulation, as listeners are more sensitive to sound inconsistencies than to minor variations in frame rate. For this reason, Nemu's overall execution speed is driven by the audio system.

Instead of running the emulator at a fixed speed and pushing audio samples to a buffer—which risks buffer underruns or overruns—Nemu uses the audio callback provided by Raylib to pull the emulation forward. When the audio system requires a new block of samples (e.g., 512 samples), it invokes a callback. This callback triggers the emulation thread to run just enough CPU, APU, and PPU cycles to generate the exact number of audio samples requested. This method ensures the emulated APU runs at the correct speed, producing perfectly synchronized audio.

A key design decision was to isolate the emulation loop on a dedicated thread, separate from the audio system. Although running the emulation logic within the audio callback is a viable approach, this separation allows control over the emulation's execution state to be transferred to the main thread. This is necessary for implementing debugging capabilities, such as pausing, resuming, or advancing the simulation on a per-cycle basis.

4 CPU Emulation

The *Central Processing Unit*—CPU—is contained within the Ricoh 2A03 and is based on the MOS Technology 6502 processor. However, Ricoh removed the 6502's binary-coded decimal (BCD) mode. Therefore, Nemu does not support BSD either. The 6502 has a CISC-based instruction set architecture (ISA) with 13 different addressing modes and 56 official instructions. Each instruction can have multiple addressing mode variants—each with a separate opcode—giving a total of 151 official opcodes. Because the instruction opcode is stored using 8 bits, there are an additional 105 unofficial opcodes. These might be utilized in ROMs and are therefore also supported by Nemu.

The addressing modes are:

- Implied: The instruction requires no operand from memory.
- Accumulator: The instruction operates on the accumulator register.
- Immediate: The operand is the byte immediately after the opcode.
- Zero Page: The operand is an 8-bit address in the zero page ($00xx).
- Zero Page X: The operand is an 8-bit address in the zero page offset by the value in register X.
- Zero Page Y: The operand is an 8-bit address in the zero page offset by the value in register Y.
- Relative: The operand is an 8-bit offset from the program counter.
- Absolute: The operand is a 16-bit address.
- Absolute X: The operand is a 16-bit address offset by the value in register X.
- Absolute Y: The operand is a 16-bit address offset by the value in register Y.
- Indirect: The operand is a 16-bit address pointing to the target address.

- Zero Page Indirect X: The operand is an 8-bit address in the zero page offset by X, pointing to the target address.
- Zero Page Indirect Y: The operand is an 8-bit address in the zero page, pointing to the target address offset by Y.

The execution of each instruction is defined by a set of properties, including its type, addressing mode, byte size, and cycle count. Within Nemu, these characteristics are statically defined in a series of lookup tables indexed by the corresponding instruction opcode. This allows for efficient decoding of instructions.

In authentic NES hardware, an instruction's effects occur over several CPU cycles. In contrast, Nemu employs a simplified timing model. The instruction is resolved immediately upon being fetched, after which the CPU is stalled for the required number of cycles. This abstraction significantly reduces implementation complexity by avoiding the need for a more granular, cycle-accurate state machine. While this approach diverges from the original hardware behavior, it is a common emulation strategy and is not expected to have any discernible impact on the execution of official game titles.

5 PPU Emulation

The *Picture Processing Unit*—PPU—is a key component of the Nintendo Entertainment System, responsible for generating the video output. For its time, the PPU was considered a sophisticated piece of hardware, making accurate emulation one of the more challenging aspects of creating an NES emulator.

The PPU renders an image as a grid of 32×30 tiles, with each tile measuring 8×8 pixels. This process involves a clear separation between background and foreground elements. The background is a static layer, while the foreground consists of movable objects known as *sprites*, such as player characters and enemies. A single sprite can be composed of one or two tiles, and a character may be represented by multiple sprites that are programmed to move in unison.

The PPU relies on multiple specific memory regions to manage video data:

- Pattern Table: All of the tile data for both backgrounds and sprites is stored in pattern tables, which are shared between the two layers. These tables are typically contained within a ROM chip on the game cartridge, but can also be implemented using RAM, which allows the data to be changed during runtime. Each tile is a 16-byte block of data that defines 64 palette offsets, one for each pixel in the tile.
- Nametable: The background is defined by nametables stored within the PPU's Video RAM (VRAM). Each nametable is a 1-kilobyte memory region that holds a list of IDs pointing to specific tiles contained within the pattern tables. VRAM can hold two nametables at a time. While the VRAM is only 2 kilobytes, it is mirrored to fill a 4-kilobyte address space, thereby creating 4 logical nametables. The mirroring is controlled by the cartridge and is used to accomplish horizontal or vertical scrolling. Additionally, each nametable contains a 64-byte attribute table. This table specifies the palette used for each tile.
- Object Attribute Memory (OAM): The foreground sprites are controlled by this 256-byte memory. It contains 64 sprite entries, each 4 bytes in size, which hold all the necessary rendering information, including the sprite's position and tile ID, among others.
- Palette RAM: This is a 32-byte memory region that holds four palettes, each with four colors. The first color of each palette serves as a transparent color, which allows the background to show through parts of a sprite. The transparent color in the first palette is also used as the global background color. Each of these colors is an index to one of 64 preconfigured colors.

CPU communication with the PPU's VRAM and OAM is handled through I/O mapped registers in the CPU's address space. The CPU can also initiate a Direct Memory Access (DMA) transfer, which efficiently copies data to the entire OAM by writing to a specific address.

The PPU's rendering process has strict timing requirements to synchronize its output with the television's electron gun. The NTSC version of the NES renders 261 scanlines per frame, with each scanline consisting of 341 cycles. Each cycle corresponds to a single pixel on the screen. This means the PPU must output the correct color for each pixel at the precise moment it is being drawn.

The PPU renders continuously, and changing its memory contents during an active render cycle will introduce visual artifacts. To prevent this, the PPU provides a safe window for memory updates called VBlank. On the first cycle of scanline 241, the PPU enters VBlank, a period of non-visible scanlines. This is the optimal time for the CPU to transfer new data to the nametables and OAM to update the background and move sprites, as only 240 scanlines and 256 cycles per scanline are visible on-screen. A DMA transfer is often used to transfer data to the OAM since individual writes are too slow to finish before the VBlank period ends.

The tight timing requirements are the most difficult aspect of PPU emulation. While a cycle-accurate emulator would account for every event at specific cycles, simpler emulators can take shortcuts to simplify implementation. Nemu does not strive for full cycle accuracy. Instead, it renders all foreground sprites on the first non-visible cycle of each visible scanline. While this may cause visual anomalies in later games that rely on intricate hardware timing, it works without issue for most titles.

6 Cartridge and Mapper Emulation

The cartridge (game pak) is the removable storage medium used to hold and run games on the console. It is a plastic casing that houses a circuit board containing the game software and, optionally, additional hardware. The circuit board includes several memory chips, including:

- PRG-ROM (Program ROM): Contains the actual code—the instructions that the CPU executes to run the game logic.
- CHR-ROM (Character ROM): Contains the game's graphical data, such as sprites for characters, and tiles for the background. Some cartridges contained CHR-RAM instead, allowing the game to load or create new graphics during runtime.
- SRAM (Static/Save RAM): Some cartridges contained a small amount of SRAM. This RAM was kept powered by a small coin-cell battery soldered to the circuit board, allowing the user the preserve their game progress on certain games.

While the 16-bit address space only allows for up to 64 KB of memory, the designers of the NES had incredible foresight and designed the system to be highly expandable, allowing cartridge manufacturers to include more memory by using memory banking. This is done through memory mappers, which are chips contained within a cartridge. A mapper correlates a specific address asserted by the CPU or PPU to some memory from a chip contained in the cartridge. Certain mappers can be configured through I/O mapped registers. They may even contain additional APU channels and trigger CPU interrupts. Nintendo itself designed and manufactured about half a dozen specific families of mapper chips called MMCs (Memory Management Controllers). However, hundreds more exist. The memory chips are produced independently of the cartridge itself, and different cartridges may contain the same mapper chip while using different memory chip sizes and ROM/RAM.

Nemu has support for NROM, MMC1, MMC2, MMC3, and MMC4.

To handle the hundreds of different cartridge configurations, Nemu uses a modular software architecture. Each mapper is implemented as a separate, specialized data structure. The main Cartridge object, which holds the game's ROM data, is generic. It contains a pointer to whichever specific mapper structure is required for the loaded game.

This design is achieved using subtype polymorphism, a software pattern that allows different data structures to be treated uniformly through a set of common functions. It is essentially a technique to achieve interface-like behavior without an interface language feature.

Nemu's cartridge object is constructed from a parsed iNES file. It is the standard format for the distribution of NES binary programs (ROMs). It is sometimes referred to as .nes (due to the file extension). Over the years, multiple derivations of the iNES header have been developed to address previous shortcomings. The latest and most commonly used today is the NES 2.0 header format, which is backwards compatible with the original iNES header. The header specifies essential parameters like the size of the ROM and RAM, the required mapper and submapper IDs, the nametable mirroring layout, and the intended TV system.

Nemu has compatibility with NES 2.0 and, by extension, iNES.

7 APU Emulation

The Audio Processing Unit—APU—is responsible for all audio synthesis on the NES. It is physically integrated into the Ricoh 2A03 chip alongside the CPU and operates at the same clock frequency. The APU generates sound through five distinct channels, which are digitally mixed and converted into a final analog audio signal. Control over these channels is managed by the CPU through a set of memory-mapped I/O registers.

The five channels each have a unique character and purpose:

- Pulse Channel 1 and 2: These two nearly identical channels produce a pulse wave, a type of square wave whose timbre can be altered by changing its duty cycle. Four duty cycle settings are available (12.5%, 25%, 50%, and 75%), giving the sound a thinner or richer quality. These channels are the workhorses of the NES soundscape, typically used for lead melodies, harmonies, and common sound effects. Each pulse channel includes:
  - An envelope generator to control the volume over time, allowing for notes that fade out.
  - A sweep unit to dynamically adjust the pitch up or down, creating effects like sirens or arpeggios.
- Triangle Channel: This channel generates a pure triangle wave. Unlike the pulse channels, it has no direct volume control; its output is either on or off at a fixed volume. Because of its smoother, less harsh tone, it is almost exclusively used for basslines, melodic counterpoints, or woodwind-like instruments. Although it lacks a volume envelope, a linear counter allows its output to be stopped after a set duration.
- Noise Channel: The noise channel produces pseudo-random static, making it ideal for percussive sounds like snares, cymbals, and hi-hats. It is also used for atonal sound effects such as explosions, engine noise, or wind. The channel can operate in two modes, generating either metallic-sounding short-period noise or a lower-pitched, rumbling long-period noise. Like the pulse channels, it includes an envelope generator for volume control.
- Delta Modulation Channel: This unique channel plays low-resolution, 1-bit delta-encoded digital audio samples (DPCM). It allows games to play back simple pre-recorded sounds, which are stored in the game's program ROM. The DMC fetches sample data from the CPU's memory space using Direct Memory Access (DMA). It was most often used for complex drum sounds, percussion fills, or short voice clips (e.g., the iconic "Fight!" in Punch-Out!!).

The digital outputs from the five channels are combined to produce the final audio waveform. On the original hardware, this mixing process is non-linear; the pulse channels are combined separately from the other three channels before being mixed, meaning the output level of one channel can affect the perceived volume of another. In Nemu, this complex interaction is efficiently simulated either through a set of pre-calculated lookup tables or a linear approximation formula.

8 Discussion

Nemu stands as a functional emulator that successfully implements the core hardware components of the Nintendo Entertainment System, including the Central Processing Unit, Picture Processing Unit, and Audio Processing Unit. The project successfully achieved its primary goals, which included support for a wide range of common mappers and NES file formats, complemented by a highly usable debug interface. However, several key areas require further development to improve accuracy and performance.

A significant effort was made to ensure the logical correctness of the CPU. The core passes a suite of standard instruction-level tests, confirming that it correctly executes individual opcodes. This forms a solid foundation for the emulator.

The most critical limitation of the current implementation is the lack of precise timing synchronization between the major system components. Due to time constraints, cycle-accurate timing for the CPU, PPU, and APU interactions was not verified. Many NES games exploit the hardware's precise timing for core gameplay logic and advanced graphical effects. The absence of this synchronization is the likely root cause of numerous observed bugs.

A prominent example of this issue lies in the implementation of Mapper 4 (MMC3). This mapper uses a scanline counter to trigger interrupts (IRQs), a technique used by games like Kirby's Adventure and Super Mario Bros 3 for sophisticated graphical effects like split-screen scrolling. For this counter to function correctly, it must be clocked with near-perfect PPU timing relative to the CPU's execution. The current implementation fails in this regard, leading to graphical glitches and incorrect behavior in these titles. Other issues regarding the PPU and APU have been observed in games like Contra and Super Mario Bros 1.

Performance optimization was not a primary focus of the initial development phase, resulting in suboptimal execution speed. Due to the emulation being driven by the audio system, the emulator will execute several cycles after which execution will be halted while waiting for the audio system to issue a new sample request. This architecture leads to significant frametime variance, where frames are not delivered at a consistent pace.

The target for smooth gameplay is a stable frametime of approximately 16.7 ms (corresponding to 60 frames per second). The current audio-driven approach makes this target unattainable. While general code optimization may yield some improvements, achieving consistent performance will likely require an architectural change, such as decoupling the main emulation loop from the audio synchronization process.

Despite the identified issues, Nemu provides a robust framework for further work. The functional debug UI is a particularly valuable asset for diagnosing and resolving the bugs outlined above. Future development should prioritize the following areas:

- Implement Cycle-Accurate Timing: The highest priority is to correctly model the timing relationship between the CPU, PPU, and APU. This will resolve a large class of compatibility issues, particularly those related to mapper IRQs and advanced graphical techniques.
- Overhaul the Main Loop for Performance: The emulation loop should be re-architected to deliver a consistent framerate, independent of audio buffering, to eliminate judder and improve the user experience.
- Enhance Compatibility: With a correct timing system in place, a systematic effort can be made to address remaining game-specific bugs and implement unimplemented hardware behaviors to broaden the library of playable titles.

9 Conclusion

In conclusion, the Nemu project successfully achieved its primary objective of developing a functional, cross-platform emulator for the Nintendo Entertainment System. The resulting application emulates the core NES hardware components with sufficient accuracy to run games developed early in the NES's lifecycle.

While Nemu suffers from frame-pacing issues and a family of emulation-related bugs and unimplemented hardware behavior, it stands as a successful proof-of-concept. It demonstrates a comprehensive understanding and application of low-level hardware architecture, memory management, and the complex timing interactions inherent in console emulation. The project not only preserves the gameplay of a beloved console but also provides a solid codebase for future enhancements aimed at achieving greater accuracy and performance.

References

[1] "NES reference guide," *Nesdev Wiki*. Accessed: Aug. 13, 2025. [Online]. Available: `https://www.nesdev.org/wiki/NES_reference_guide`

[2] "MOS 6502," *CPC Wiki*. Accessed: Aug. 13, 2025. [Online]. Available: `https://www.cpcwiki.eu/index.php/MOS_6502`

A Images

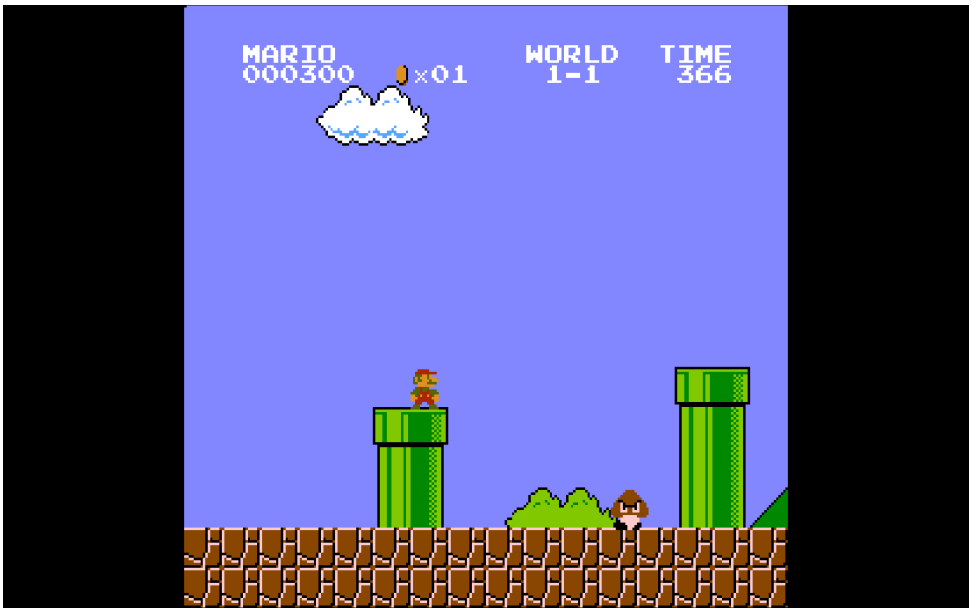

Figure 1: Nemu running Super Mario Bros 1

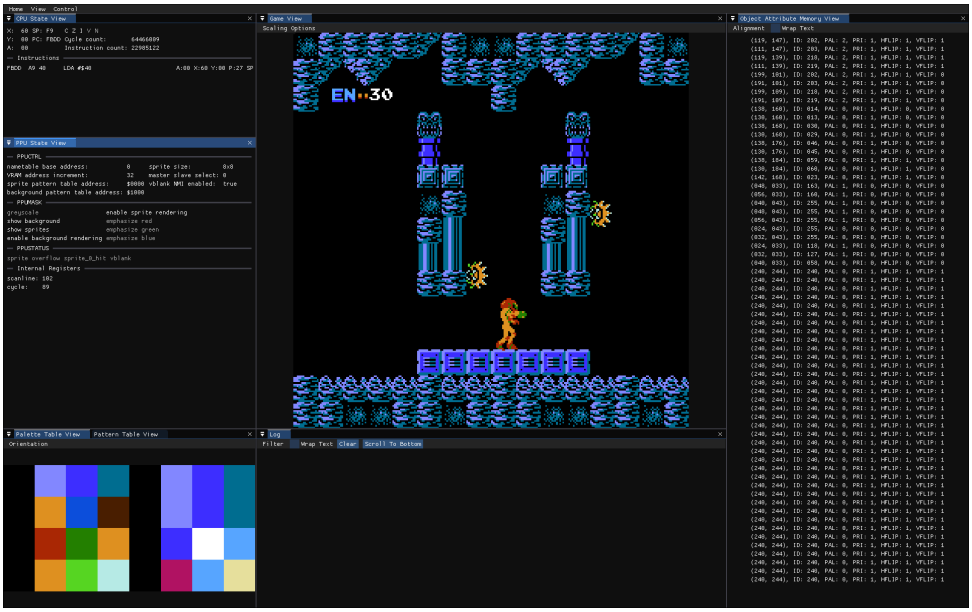

Figure 2: Nemu running Metroid with the debug UI activated